

INTERNSHIP PROJECT REPORT

WorkPulse: Enterprise Employee Management System

A Full-Stack Cloud-Native Human Resource & Workforce Operations Platform

Project Domain:	Enterprise Full-Stack Web Development & Operations
Technology Stack:	Spring Boot 3, Java 17, PostgreSQL, React 18, Material UI, Docker
Architecture:	RESTful Micro-Ready Monolith + SPA + Cloud Micro-Services
Cloud Infrastructure:	Render (Backend & Database) + Vercel (Frontend Hosting)
GitHub Repository:	https://github.com/Agrim027/WorkPulse-Employee-Management-System
Production URL:	https://workpulse-em-system.vercel.app
Backend API Endpoint:	https://workpulse-backend-c6a7.onrender.com/api/v1
Document Version:	1.0.0 (Final Internship Submission)

Author / Intern: Software Engineering Intern

Submission Date: August 19, 2026

Status: Production Live & Fully Verified

1. Executive Summary

WorkPulse is a cloud-native, full-stack enterprise Employee Management System (EMS) designed to streamline corporate human resource workflows, organizational structure management, daily attendance logging, and monthly payroll operations.

During this internship, the primary objective was to architect, develop, containerize, and deploy a production-grade Web application leveraging **Spring Boot 3 (Java 17)** for backend services, **Render PostgreSQL** for persistent data storage, and **React 18 (Vite + Material UI)** for a responsive Single Page Application (SPA) frontend interface.

The application incorporates industry-standard security practices, including JWT-based Stateless Authentication, Role-Based Access Control (RBAC with `ROLE_ADMIN`, `ROLE_HR`, and `ROLE_EMPLOYEE`), strict CORS origin pattern matching, automated programmatic database seeding, and production Single Page Application routing fallbacks on Vercel.

Key Highlight	Implementation Summary
Role-Based Access Control	Granular authorization enforcing Admin, HR, and Employee privileges across UI & APIs.
Department Management	CRUD operations for department structures, codes, and organizational units.
Attendance Tracking	Self-service Check-In/Check-Out, daily logging, multi-criteria filtering, and status chips.
Payroll & Compensation	Automatic Net Salary calculations (Basic + Allowances - Deductions) and monthly slip management.
Cloud CI/CD & Hosting	Automated builds via Render Docker containers and Vercel static asset deployment.

2. System Architecture & Tech Stack

WorkPulse follows a classic multi-tier client-server architecture cleanly separating presentation, business logic, security, and persistence layers:

Layer	Technology Selection	Key Responsibilities
Frontend (Client)	React 18, Vite, Material UI (MUI v5), Axios, React Router 6	Single Page Application (SPA), state management, responsive UI dashboards, JWT storage.
Backend (API)	Java 17, Spring Boot 3.x, Spring Data JPA, Lombok	RESTful APIs, business validation, DTO mapping, pagination, transactions.
Security Layer	Spring Security 6, JWT (io.jsonwebtoken 0.11.5), BCrypt	Stateless authentication, Bearer JWT validation, password hashing, CORS filters.

Database	Render PostgreSQL (Managed Service)	Relational data storage, foreign key constraints, transactional consistency.
DevOps & Cloud	Docker, Render Web Services, Vercel Edge Network	Multi-stage container builds, automated Git push deployments, SPA rewrite routing.

3. Core Modules & Key Features

3.1 Role-Based Operational Dashboards

WorkPulse presents tailored dashboard views depending on user authorities:

- **Admin / HR Dashboard:** Displays organization-wide summaries including Total Employees (Active vs Terminated count), Department structural metrics, monthly payroll totals, and company-wide attendance breakdown (Present, Absent, Leave). Uses GET `/api/v1/dashboard/summary`.
- **Employee Personal Dashboard:** Displays personal quick-action Check-In / Check-Out triggers, recent attendance history, and personal salary slip summaries using GET `/api/v1/dashboard/my-summary`.

3.2 Department Management Module

Allows administrators and HR managers to define organizational departments (e.g., Engineering, Human Resources, Finance). Enforces unique department codes, description fields, and referential integrity protection to prevent deletion of departments currently assigned to active employees.

3.3 Employee Profile Directory

Centralized directory managing employee personal information (Name, Email, Phone, DOB, Gender, Address), employment details (Employee Code, Department, Joining Date), and employment status (ACTIVE, INACTIVE, ON_LEAVE, TERMINATED). Supports full-text search, department filtering, pagination, and soft deletion to preserve historical payroll data.

3.4 Attendance & Daily Time Tracking

Supports self-service employee Check-In / Check-Out for the current date, as well as HR administrative logging. Features single-date, date-range, and status filtering (PRESENT, ABSENT, HALF_DAY, LEAVE).

3.5 Payroll & Compensation Management

Enables HR administrators to generate monthly salary slips for employees with Basic Salary, Allowances, and Deductions. Net Salary is authoritatively calculated on the backend as Basic + Allowances - Deductions. Supports payment status tracking (PENDING, PAID, CANCELLED).

4. Database Schema & Data Models

The relational database schema is structured around 6 key JPA entities connected via foreign key relationships:

Entity Name	Table Name	Primary Keys & Foreign Keys	Key Attributes & Enum Constraints
User	users	PK: id FK: user_roles (M2M)	username (unique), email (unique), password (BCrypt), enabled.
Role	roles	PK: id	name (ERole: ROLE_ADMIN, ROLE_HR, ROLE_EMPLOYEE), description.
Department	departments	PK: id	department_code (unique), name, description, created_at.
Employee	employees	PK: id FK: user_id (1to1), department_id (Mto1)	employee_code (unique), first_name, last_name, email, employment_status.
Attendance	attendances	PK: id FK: employee_id (Mto1)	attendance_date, check_in, check_out, status (EAttendanceStatus), remarks.
Salary	salaries	PK: id FK: employee_id (Mto1)	salary_month, basic_salary, allowances, deductions, net_salary, payment_status.

5. Key Engineering Challenges & Solutions Resolved

Challenge / Problem	Root Cause Analysis	Resolution Implemented
Production CORS Preflight Failure	Chrome blocked login requests due to strict single-origin mismatch between Vercel dynamic preview URLs and Render backend.	Updated CorsConfig.java to parse comma-separated origins and use setAllowedOriginPatterns for pattern matching (https://*.vercel.app). Explicitly permitted HTTP OPTIONS in SecurityConfig.java.
Vercel SPA 404 Route Errors	Direct navigation or refreshing routes like /dashboard or /login returned Vercel 404 because static file server looked for real directory files.	Created frontend/vercel.json specifying clean SPA rewrite rules directing all dynamic route requests back to /index.html.
Admin User Role Assignment	Default registration assigned ROLE_EMPLOYEE to user admin, blocking access to the Admin Dashboard.	Created DataInitializer.java implementing CommandLineRunner to automatically seed roles and upgrade admin to ROLE_ADMIN on backend boot. Updated AuthServiceImpl for registration.

Department Creation Table Refresh Bug	UI showed 'Department created successfully' toast but table rendered 'No departments found' because code evaluated <code>res.data.content</code> instead of <code>res.data.data</code> .	Systemically refactored all frontend pages and modals to safely unpack <code>res.data.data</code> from the backend's standard <code>ApiResponse</code> container.
--	--	---

6. Production Deployment & Cloud Infrastructure

6.1 Backend Docker Containerization

The Spring Boot backend utilizes a multi-stage Dockerfile to optimize image size and build speed:

```
# Stage 1: Build JAR using Maven
FROM eclipse-temurin:17-jdk-alpine AS builder
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

# Stage 2: Minimal Execution Runtime
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

6.2 Cloud Deployment Verification

- **Production Backend Service:** <https://workpulse-backend-c6a7.onrender.com> (Render Web Service)
- **Managed Production Database:** Render PostgreSQL Service named `workpulse-db`
- **Production Frontend Application:** <https://workpulse-em-system.vercel.app> (Vercel Global CDN)

7. Conclusion & Future Scope

The WorkPulse Internship Project successfully met all deliverables, resulting in a production-ready, cloud-deployed Employee Management System. Key takeaways include mastering multi-tier architecture design, securing Spring Boot applications with stateless JWT and CORS policies, containerizing Java micro-services with Docker, and building production React SPAs with Material UI.

Future Roadmap Enhancements:

1. **Automated Payslip PDF Downloads:** Allow employees to download PDF salary slips directly from their dashboard.
2. **Leave Application Workflow:** Implement a formal leave request and approval flow between Employees and HR.
3. **Analytical Export:** Export attendance and payroll reports in CSV / Excel formats for accounting teams.